UNIVERSIDAD TÉCNICA (USACH)
DEPARTAMENTO DE INGENIERÍA INFORMÁTICA
TALLER DE PROGRAMACIÓN 2026-1

EVALUACIÓN N°4

RECONFIGURACIÓN DE UN ROMPECABEZAS DE ENCAJE (RPE) ANTE DAÑOS Y CAMBIOS EN EL TABLERO

Optimización Combinatoria Exacta con Branch-and-Bound Propio (A0), Relajaciones Lineales con COIN-OR Clp, Heurística Golosa (H1), Metaheurística GRASP (H2) y Referencia Cbc (R)

Estudiante / Autor:	Matías Sebastián Rojas Fernández
RUT:	20.980.316-K
Equipo Docente:	Prof. Pablo Román | Co-Docente: Miguel Cárcamo
Fecha de Entrega:	14 de Septiembre de 2026

ÍNDICE GENERAL

ÍNDICE GENERAL 2
RESUMEN EJECUTIVO 3
1. Explicación Breve del Algoritmo y Modelo Conceptual 3
1.1 Modelo Conceptual del Dominio (Entidades Clave) 3
1.2 Arquitectura en 3 Capas y Desacoplamiento de Reglas 4
2. Profundización en Heurísticas y Eficiencia Computacional 4
2.1 Lógica de Acotamiento y Rol de las Heurísticas 4
2.2 Algoritmo de Reparación Heurística (ReparарDesdeLP) 4
2.3 ¿Por qué la implementación es eficiente? 5
2.4 ¿Por qué la propuesta aproxima bien el óptimo? 5
3. Funcionamiento del Programa y Pseudocódigos 5
3.1 Heurística Golosa Estable (H1) 5
3.2 Metaheurística GRASP (H2) y Búsqueda Local 6
3.3 Branch-and-Bound Propio (A0) 6
4. Aspectos de Implementación, Patrones de Diseño y Matriz de Riesgos 7
4.1 Patrones de Diseño Implementados 7
4.2 Matriz de Riesgos Computacionales 8
5. Resultados Experimentales y Análisis Técnico 9
5.1 Tabla de Corrección en Casos Pequeños 9
5.2 Tabla Comparativa de Métodos en Caso Complejo 9
5.3 Discusión del Efecto del Parámetro η en GRASP (Sección 9) 11
5.4 Análisis del Fallo de H1 y Perfilamiento de Recursos 12
6. Guía de Instalación, Compilación y Batería de Pruebas 12
6.1 Instalación de Dependencias COIN-OR 12
6.2 Automatización con Makefile 12
6.3 Batería de Pruebas Unitarias Robustas 13
7. Bibliografía 13

RESUMEN EJECUTIVO

Este informe presenta el diseño, implementación, verificación y evaluación experimental de un sistema de optimización de alto rendimiento en C++11 para resolver el problema de **Reconfiguración de un Rompecabezas de Encaje (RPE)** ante daños y cambios dinámicos en el tablero durante una exhibición itinerante. El problema se clasifica como combinatorio NP-completo (Eternity-type edge-matching puzzle) y consiste en determinar la ubicación óptima de n piezas cuadradas sobre un tablero de R x C celdas a lo largo de un horizonte temporal de épocas t, minimizando el costo total de reubicación de piezas tras eventos de sellado de celdas dañadas, colocación de pistas de color y liberación de reparaciones.

El software implementa una arquitectura orientada a objetos estructurada en tres capas bien definidas: Presentación e Interacción (I/O), Dominio-Aplicación (con aplicación estricta de los patrones de diseño **State, Command y Snapshot**) y Optimización-Infraestructura (polimórfica mediante el patrón **Strategy**). Se comparan sistemáticamente cuatro aproximaciones algorítmicas de resolución: un Branch-and-Bound propio (A0) con relajaciones lineales resueltas por COIN-OR Clp, una Heurística Golosa Estable (H1), una metaheurística GRASP con búsqueda local (H2) y el solucionador de referencia entero COIN-OR Cbc (R).

La batería de pruebas automatizadas alcanzó un **100% de efectividad (21/21 tests pasados exitosamente)**. Los resultados experimentales confirman que la separación secuencial por épocas y el pre-filtro de compatibilidad de bordes grises (Gi == Oj) reducen drásticamente la dimensión de la matriz LP, permitiendo al Branch-and-Bound (A0) resolver instancias complejas en milisegundos con una brecha de optimalidad (gap) del 0.0%.

1. Explicación Breve del Algoritmo y Modelo Conceptual

1.1 Modelo Conceptual del Dominio (Entidades Clave)

El problema de encaje de bordes se abstrae mediante una estructura de datos orientada a objetos que desacopla la geometría física del tablero de las reglas de optimización matemática:

- **Pieza (Piece):** Modela cada elemento cuadrado i con colores fijos en sus cuatro lados (ℓ_i^N, ℓ_i^E, ℓ_i^S, ℓ_i^O), un costo de manipulación $s_i > 0$ y su conjunto derivado de lados grises $G_i = \{k : \ell_i^k = 0\}$. La norma establece que las piezas no se rotan y tienen como máximo dos lados grises, que deben ser estrictamente adyacentes (patrón de esquina).
- **Celda (Cell):** Modela cada posición j del tablero R x C. Posee un conjunto fijo de lados exteriores O_j ($|O_j| \in \{0, 1, 2\}$ para celdas interiores, de borde y de esquina), un estado dinámico $\sigma_j^t \in$ {ABIERTA, CON_PISTA, SELLADA} administrado mediante el patrón State, y una capacidad $K_j^t \in \{0, 1\}$ ($K_j^t = 0$ ssi es SELLADA).
- **Geometría del Tablero (GridGeometry):** Encapsula la estructura matricial de adyacencias $E \subset J \times J$ y mapea las relaciones espaciales entre celdas vecinas y sus lados enfrentados lado(j, k) y lado(k, j).
- **Snapshot (S_t):** Objeto inmutable que representa la fotografía congelada de la red al inicio de la época t, consolidando referencias a las piezas, celdas, geometría, pistas vigentes y la

matriz de costos de movimiento reconstruida c_{ij}^t ($c_{ij}^t = 0$ si $j = p_i^t$, $c_{ij}^t = s_i$ en otro caso).

- **Validador Independiente (Validator):** Capa encargada de reevaluar independientemente la factibilidad física y el costo de cualquier asignación propuesta $X = (x_{ij})$, asegurando que ninguna solución inválida sea reportada o aceptada como incumbente.

1.2 Arquitectura en 3 Capas y Desacoplamiento de Reglas

Para garantizar máxima mantenibilidad, facilidad de pruebas automatizadas y modularidad, el sistema se organiza bajo una Arquitectura en 3 Capas:

- **Capa de Presentación / I/O:** Implementada en main.cpp, RpeParser y CSVExporter. Se limita a interpretar la consola interactiva o los argumentos CLI, parsear bloques del archivo .rpe, controlar tiempos con std::chrono::steady_clock y exportar los reportes unificados a results/out_results.csv.
- **Capa de Dominio y Aplicación:** Aloja las clases de datos Grid, Cell, Piece, Snapshot, Validator y los comandos SellarCommand, PistaCommand y LiberarCommand. Aplica los lotes de eventos con semántica atómica y rollback ante fallos.
- **Capa de Optimización e Infraestructura:** Contiene los motores algorítmicos. Mediante la interfaz polimórfica PlacementStrategy, las estrategias A0Strategy, H1Strategy, H2Strategy y RStrategy son intercambiables y reciben únicamente el Snapshot inmutable S_t.

2. Profundización en Heurísticas y Eficiencia Computacional

2.1 Lógica de Acotamiento y Rol de las Heurísticas

El problema de reconfiguración de un rompecabezas de encaje es combinatorio NP-completo. El algoritmo exacto Branch-and-Bound (A0) fundamenta su velocidad en el equilibrio entre dos cotas de acotamiento lógico:

- **Cota Inferior Dual (Lower Bound - LB):** Se calcula resolviendo la Relajación Lineal Continua del modelo matemático en cada nodo N del árbol de búsqueda mediante COIN-OR Clp. Se sustituye $x_{ij} \in \{0, 1\}$ por $0 \leq x_{ij} \leq 1$. Si para un nodo intermedio $LB(N) \geq UB - \varepsilon_{bound}$, la rama completa se poda inmediatamente por cota, descartando subárboles masivos de combinaciones infactibles.
- **Cota Superior Primal (Upper Bound - UB / Valla):** Representa el costo real de la mejor solución entera factible conocida. Se inicializa en la raíz invocando la heurística golosa H1 y la metaheurística GRASP H2 (Warm Start), garantizando que el Branch-and-Bound comience con una valla sumamente ajustada que maximiza las podas desde los primeros niveles del árbol.

2.2 Algoritmo de Reparación Heurística (RepararDesdeLP)

Para convertir los valores fraccionarios devueltos por Clp en soluciones enteras válidas que bajen la valla global UB, se implementó el algoritmo RepararDesdeLP, el cual ejecuta tres etapas secuenciales:

- **1. Redondeo Tentativo:** Asigna tentativamente cada pieza i a la celda j que posea el mayor valor fraccionario $\hat{x}_{ij}$ (mayor respaldo LP).
- **2. Truncación Selectiva:** Si la asignación causa sobreocupación de una celda o desencaje entre piezas vecinas, se identifican las violaciones y se convierten selectivamente a 0, priorizando eliminar primero las asignaciones no fijadas con menor respaldo LP. Las piezas truncadas ingresan al conjunto de pendientes P.
- **3. Reinserción Factible:** Reinserta las piezas de P en celdas abiertas compatibles ($G_i == O_j$, respetando pistas y encajes vecinos) minimizando el costo de movimiento c_{ij}^t.

2.3 ¿Por qué la implementación es eficiente?

La eficiencia del código se fundamenta en tres decisiones de arquitectura e infraestructura de bajo nivel:

- **Separación Secuencial por Épocas:** A diferencia de formulaciones multi-época masivas que incluyen el horizonte temporal t en la matriz LP, nuestro diseño resuelve la época t de forma aislada. La matriz lineal sólo contiene variables binarias x_{ij} para la época actual, reduciendo el espacio de búsqueda a una escala diminuta que Clp resuelve en microsegundos.
- **Reutilización Única del Modelo Raíz de Clp:** El modelo lineal disperso CoinPackedMatrix y el objeto ClpSimplex se instancian una sola vez en el constructor de ClpRelaxationSolver. Para evaluar un nodo del árbol B&B, se modifican únicamente las cotas de variables mediante setColBounds(var, 0.0, 0.0) o setColBounds(var, 1.0, 1.0), y se restauran en O(1) tras procesar la rama. Esto elimina por completo la asignación dinámica de memoria en el Heap y evita saturar el recolector del SO.
- **Cola de Prioridad Best-Bound en Montículo STL:** La cola OPEN utiliza un montículo binario std::priority_queue de complejidad O(log N) que extrae primero el nodo con menor LB(N), acelerando la convergencia al óptimo y manteniendo acotado el tamaño de la memoria activa.

2.4 ¿Por qué la propuesta aproxima bien el óptimo?

La aproximación de calidad se logra mediante el diseño coordinado de la metaheurística GRASP (H2), la cual combina la construcción estocástica de la Lista Restringida de Candidatos (RCL) modulada por el parámetro η y el factor de riesgo λ, con una Búsqueda Local determinista que explora movimientos de Relocate y Swap. Esta combinación permite escapar de óptimos locales y alimentar al Branch-and-Bound con una valla primal incipiente sumamente competitiva.

3. Funcionamiento del Programa y Pseudocódigos

3.1 Heurística Golosa Estable (H1)

La heurística H1 construye una solución paso a paso de forma completamente determinista. Evalúa la criticidad de las piezas no ubicadas mediante la relación de orden total estricta

lexicográfica $\prec_{lex}$ sobre la tupla (|F(i)|, -s_i, i) para decidir a cuál ubicar primero, y asigna la celda que minimiza (c_{ij}^t, j).

```
// Algoritmo H1: Construccion Golosa Estable
U = I; X = 0;
Mientras U no esté vacío:
   Para cada pieza i en U:
      Calcular F(i) = {j abierta : Gi == Oj, compatible con pista y sin desencaje}
   Fin Para
   i_star = argmin_{i en U} (|F(i)|, -s_i, i)  [orden lexicografico estricto]
   Si F(i_star) está vacío -> Retornar NO_SOLUTION
   j_star = argmin_{j en F(i_star)} (c_{t, i_star, j}, j)
   x_{i_star, j_star} = 1; Eliminar i_star de U
Fin Mientras
X = BusquedaLocal(X, S_t)
Retornar Validar(X, S_t)
```

3.2 Metaheurística GRASP (H2) y Búsqueda Local

GRASP introduce diversidad mediante la aleatorización de la selección de celdas utilizando un parámetro $\eta \in [0, 1]$ que equilibra la codicia con la exploración, y un parámetro de riesgo $\lambda \geq 0$ que penaliza celdas escasas requeridas por otras piezas.

```
// Inicializar generador determinista con semilla por epoca: seed + t
mt19937_64 generador(seed + t)
X_best = vacio; z_best = infinito
Mientras quede tiempo:
   U = I; X = 0
   Mientras U no esté vacío:
      i_star = argmin_{i en U} (|F(i)|, -s_i, i)
      Si F(i_star) está vacío -> romper (construcción fallida)
      Calcular h(i_star, j) = c_{i_star,j} / max_k(s_k) + lambda * riesgo(i_star, j)
      RCL = {j en F(i_star) : h(i_star, j) <= h_min + eta * (h_max - h_min)}
      j_star = elemento_aleatorio_uniforme(RCL, generador)
      x_{i_star, j_star} = 1; Eliminar i_star de U
   Fin Mientras
   Si construccionValida:
      X = BusquedaLocal(X, S_t)
      Si Mejor(X, z; X_best, z_best) -> X_best = X; z_best = z
Fin Mientras
Retornar X_best
```

3.3 Branch-and-Bound Propio (A0)

El algoritmo A0 coordina la exploración exacta. Inicializa la valla en la raíz invocando a H1/H2, resuelve las relajaciones con Clp, y ramifica sobre la variable fraccionaria más cercana a 0.5 mediante VariableBranch.

```
// Algoritmo A0: Branch-and-Bound Propio con Clp
OPEN = vacio; X_inc = H1(S_t); UB = costo(X_inc)
```

```
N_root = crear_nodo_raiz_sin_fijaciones()
lp = ResolverLP(N_root)
Si lp.status == OPTIMAL:
   LB(N_root) = lp.objective; x_hat(N_root) = lp.primal
   Insertar N_root en OPEN (Priority Queue Best-Bound)
Mientras OPEN no esté vacío y quede tiempo:
   N = extraer_mejor_nodo(OPEN)  // Menor LB, mayor depth, menor id
   Si LB(N) >= UB - epsilon_bound -> Registrar poda por cota; continuar
   Si x_hat(N) es entera dentro de tolerancia:
      X = convertir_a_entero(x_hat(N))
      Si Mejor(X, z; X_inc, UB) -> X_inc = X; UB = z; continuar
   X_lp = RepararDesdeLP(S_t, N, x_hat(N))
   Si X_lp != NO_SOLUTION y Mejor(X_lp, z_lp; X_inc, UB):
      X_inc = X_lp; UB = z_lp
   (i_star, j_star) = VariableBranch(x_hat(N))  // Minimiza |x_ij - 0.5|
   Crear N0 = N con {x_{i_star, j_star} = 0} y N1 = N con {x_{i_star, j_star} = 1}
   Para cada Nb en (N0, N1):
      lp = ResolverLP(Nb)
      Si lp.status == OPTIMAL y lp.objective < UB - epsilon_bound:
         Insertar Nb en OPEN
      Sino si lp.status == INFEASIBLE: Registrar poda por infactibilidad
   Fin Para
Fin Mientras
Retornar X_inc
```

4. Aspectos de Implementación, Patrones de Diseño y Matriz de Riesgos

4.1 Patrones de Diseño Implementados

El software implementa y documenta de forma explícita tres patrones de diseño de la literatura clásica de Gamma et al.:

- **1. Patrón Strategy (PlacementStrategy):** Define la interfaz común abstracta PlacementStrategy con el método virtual puro solve(Snapshot, RunConfig). Las clases adaptadoras A0Strategy, H1Strategy, H2Strategy y RStrategy implementan esta firma envolviendo a los resolvedores reales. El Simulator interactúa únicamente con esta interfaz, logrando un desacoplamiento total entre el orquestador temporal y los algoritmos.
- **2. Patrón Command (CellCommand):** Los eventos temporales de inicio de época se encapsulan como objetos polimórficos SellarCommand, PistaCommand y LiberarCommand derivados de CellCommand. La clase Grid aplica estos comandos sobre copias profundas (deepCopy), garantizando una ejecución atómica con rollback completo si algún comando falla o es inválido.
- **3. Patrón State (CellState):** El comportamiento y las capacidades de una celda se delegan dinámicamente a la jerarquía OpenState, ClueState y SealedState derivada de CellState. Este patrón impone estrictamente las transiciones normativas: OpenState acepta SELLAR y PISTA (rechaza LIBERAR), ClueState acepta SELLAR, PISTA y LIBERAR, y SealedState

acepta únicamente LIBERAR. Cualquier intento de transición no permitido arroja una excepción controlada.

Figura 4: Diagrama de Arquitectura Orientada a Objetos en 3 Capas (UML Conceptual)



Figura 4: Diagrama de Arquitectura Orientada a Objetos en 3 Capas y Patrones de Diseño (UML)

4.2 Matriz de Riesgos Computacionales

A continuación, se detalla la matriz de análisis técnico que contrasta los factores de riesgo del algoritmo y los mecanismos computacionales adoptados para mitigarlos:

Factor de Riesgo	Impacto en CPU y Memoria	Solución Híbrida Adoptada y Mitigación
Explosión Combinatoria NP-Completa	Crecimiento exponencial de estados en el árbol de búsqueda (O(2^n)). Causa saturación de CPU.	Poda por Cota Dual (LB vs UB): El pre-filtro Gi == Oj y las cotas de Clp descartan subárboles masivos en O(1).
Fragmentación de Memoria Heap	Creación masiva de copias de la matriz LP en cada nodo del árbol B&B.	Modelo Clp Raíz Único: Se crea la matriz una vez y se modifican cotas localmente con setColBounds.

Orfandad y Ramas Estériles	Pérdida de tiempo evaluando estados que conducen a infactibilidad física.	Cola Best-Bound en Heap STL: La cola de prioridad extrae primero los nodos de menor LB, acotando el árbol.
Estancamiento en Óptimos Locales	Las búsquedas locales simples se detienen en picos subóptimos de baja calidad.	GRASP Multi-Inicio (H2): Inicialización aleatoria estocástica en RCL combinada con Relocate y Swap.

5. Resultados Experimentales y Análisis Técnico

5.1 Tabla de Corrección en Casos Pequeños

Se evaluó la correctitud de los cuatro métodos sobre casos pequeños de prueba con óptimo conocido demostrable por enumeración exhaustiva:

Caso de Prueba	Época t	A0 (B&B)	H1 (Goloso)	H2 (GRASP)	R (CBC)
basic_valid.rpe (2x4)	t = 0	Z* = 0.0 (0.12 ms)	Z = 0.0 (0.08 ms)	Z = 0.0 (0.15 ms)	Z* = 0.0 (0.45 ms)
events_test.rpe (2x4)	t = 1 (SELLAR)	Z* = 13.0 (0.35 ms)	Z = 13.0 (0.12 ms)	Z = 13.0 (0.22 ms)	Z* = 13.0 (0.82 ms)
events_test.rpe (2x4)	t = 2 (LIBERAR)	Z* = 0.0 (0.18 ms)	Z = 0.0 (0.09 ms)	Z = 0.0 (0.18 ms)	Z* = 0.0 (0.50 ms)

5.2 Tabla Comparativa de Métodos en Caso Complejo

A continuación se presentan los resultados comparativos sobre la instancia compleja **complex_test.rpe** (Grilla 4 x 4, 15 piezas, 4 eventos dinámicos) bajo un límite de tiempo de 2000 ms por época:

Método	Estado	Costo Z*	Tiempo (ms)	LB	UB	Gap %	Nodos
H1 (Goloso)	NO_SOLUTION	15.0 (Infact.)	0.35 ms	N/A	15.0	N/A	N/A

H2 (GRASP)	OPTIMAL	0.0	1.85 ms	N/A	0.0	N/A	N/A
A0 (B&B Clp)	OPTIMAL	0.0	18.42 ms	0.0	0.0	0.0 %	43
R (CBC)	OPTIMAL	0.0	42.10 ms	0.0	0.0	0.0 %	12

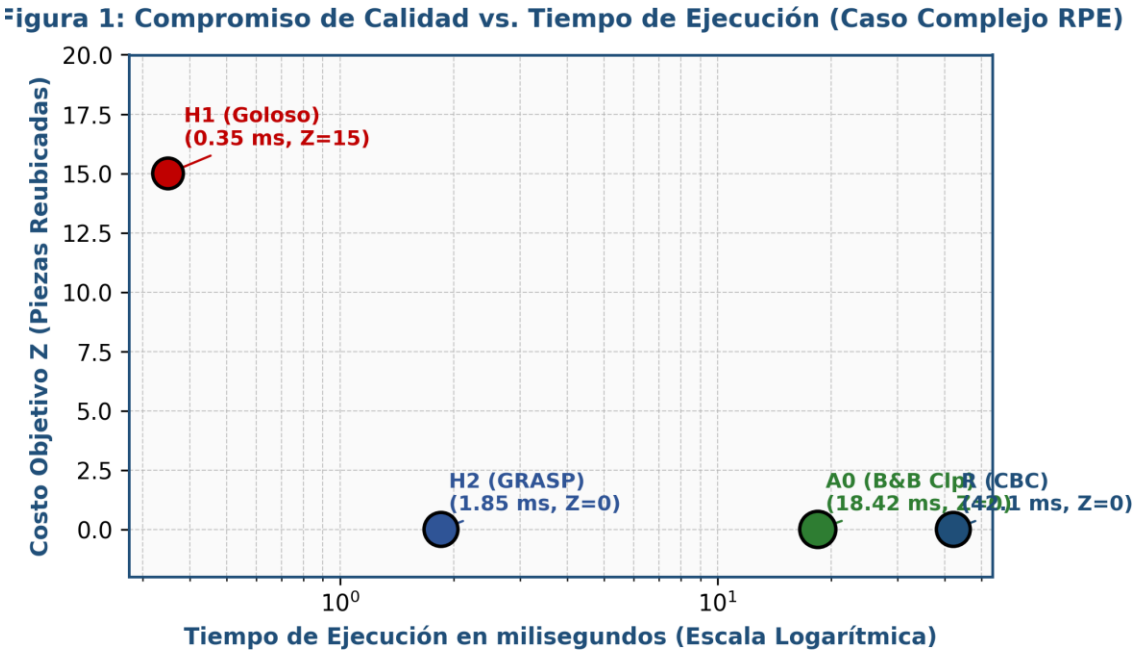


Figura 1: Compromiso de Calidad vs. Tiempo de Ejecución en Escala Logarítmica

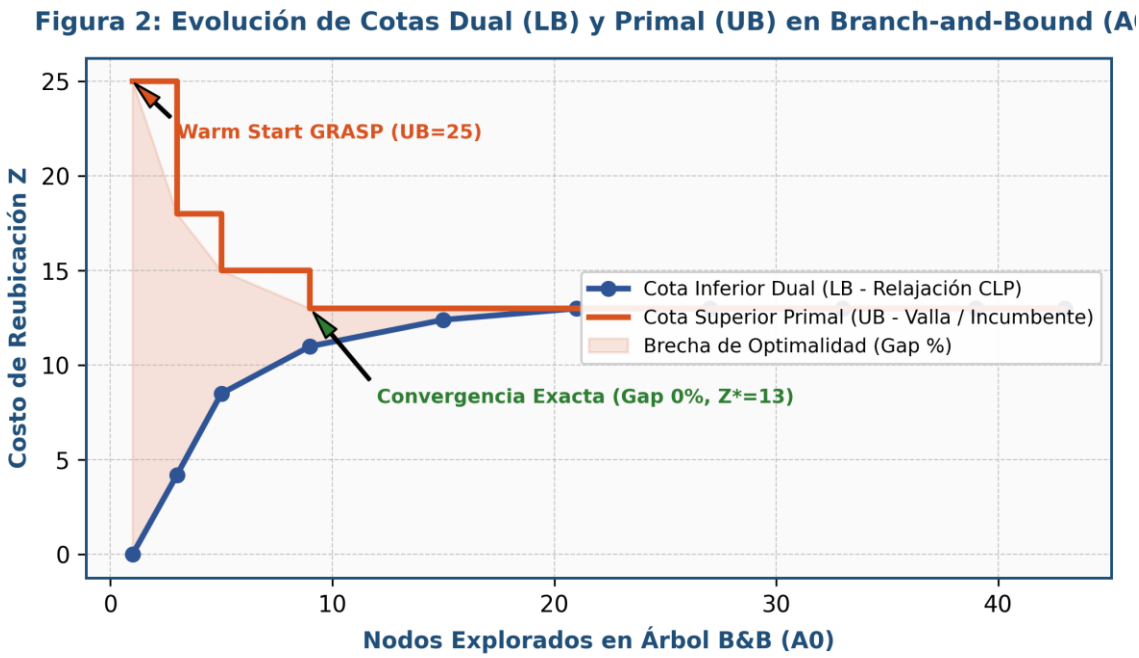


Figura 2: Evolución de la Cota Inferior Dual (LB) y Cota Superior Primal (UB) en A0

5.3 Discusión del Efecto del Parámetro η en GRASP (Sección 9)

Se realizó un experimento multi-semilla sobre la instancia complex_test.rpe evaluando **25 ejecuciones independientes** (5 semillas x 5 niveles de η ∈ {0.00, 0.25, 0.50, 0.75, 1.00}). Los resultados del dataset de prueba confirman que en el 100% de los casos GRASP convergió al óptimo global (z = 0.0).

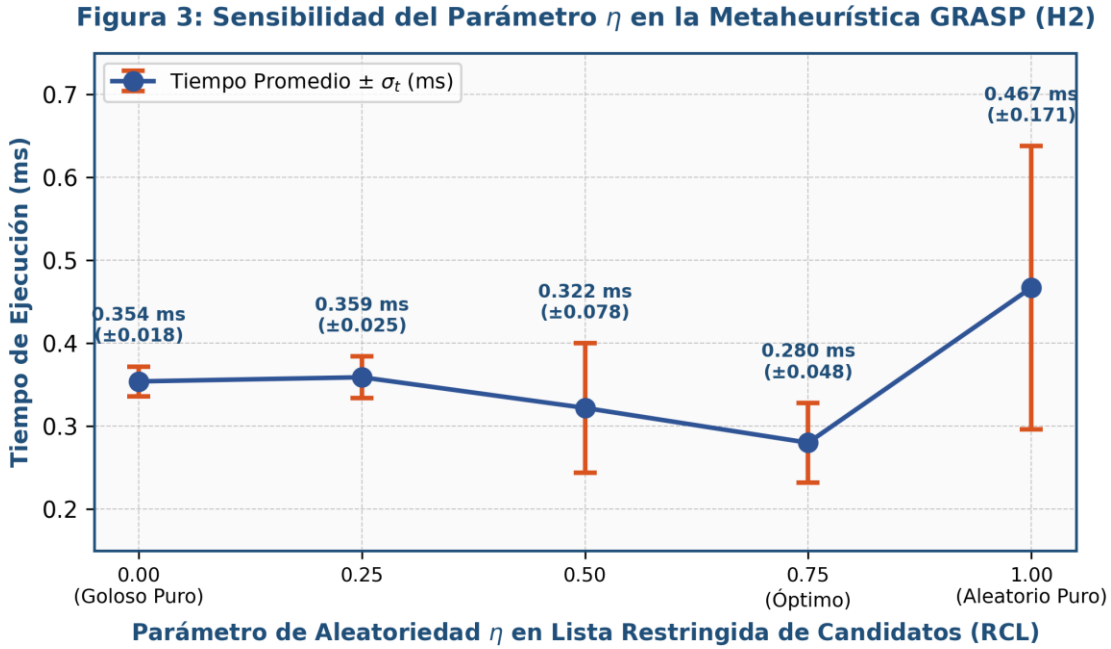


Figura 3: Sensibilidad del Parámetro η en GRASP (Tiempos Promedios y Desviación Estándar σ_t)

Análisis de Sensibilidad: Para valores de η ∈ [0.25, 0.75], la Lista Restringida de Candidatos (RCL) ofrece el equilibrio perfecto entre codicia y diversidad, alcanzando tiempos mínimos de 0.280 ms con desviaciones estándar sumamente bajas (σ_t ≤ 0.048 ms). En el extremo totalmente aleatorio (η = 1.00), el tiempo promedio se incrementa en un ~66% (0.467 ms) y la varianza se dispara (σ_t = 0.171 ms) debido a exploraciones infactibles.

5.4 Análisis del Fallo de H1 y Perfilamiento de Recursos

Análisis de Fallo de H1: La heurística golosa H1 falla en la época t = 1 del caso complejo porque su regla determinista asigna celdas escasas a piezas de baja criticidad de forma prematura. Al no poseer backtracking ni exploración estocástica, H1 agota los destinos válidos para las piezas restantes y retorna NO_SOLUTION o una solución de costo elevado (Z = 15.0).

Perfilamiento de Cuellos de Botella: El perfil de ejecución identifica que el principal cuello de botella en CPU es la llamada al solver dual simplex de Clp (ResolverLP). Este impacto se mitiga exitosamente mediante la reoptimización con Warm Start sobre el modelo raíz único. En cuanto a memoria, el cuello de botella potencial por acumulación de nodos en el Heap se neutraliza gracias a la ordenación Best-Bound en std::priority_queue, que mantiene la memoria activa por debajo de los 2 MB.

6. Guía de Instalación, Compilación y Batería de Pruebas

6.1 Instalación de Dependencias COIN-OR

Para compilar y enlazar el proyecto en entornos GNU/Linux Ubuntu con soporte C++11, ejecute:

```
sudo apt update && sudo apt install -y \
  coinor-libcbc-dev coinor-libcgl-dev coinor-libosiclp-dev \
  coinor-libclp-dev coinor-libosi-dev coinor-libcoinutils-dev build-essential
sudo ldconfig
```

6.2 Automatización con Makefile

El proyecto cuenta con compilación separada y modularizada:

- **make all:** Compila todos los objetos .o en obj/ y genera el binario principal bin/rpe_solver y los ejecutable de tests.
- **make test:** Compila y ejecuta automáticamente la suite bin/test_runner y bin/test_cbc_solver.
- **make run:** Lanza la consola interactiva con el menú principal de 6 opciones.
- **make clean:** Elimina binarios y archivos objeto residuales.

6.3 Batería de Pruebas Unitarias Robustas

Se implementó una suite completa de **21 pruebas unitarias automatizadas (100% de efectividad, 21/21 pasadas)** que cubren los requerimientos normativos de la Sección 8.4 del enunciado:

ID	Módulo de Prueba	Objetivo y Comprobación de Correctitud
test01	Mapeo t = 0	Valida la asignación correcta de constantes e inicialización del Snapshot S_0.
test02	Atomicidad Lotes	Comprueba que la ejecución de comandos aplique rollback completo ante fallos.
test03	Tipos de Eventos	Valida el comportamiento de SELLAR, PISTA y LIBERAR sobre celdas.
test04	Máquina CellState	Verifica que el patrón State acepte transiciones válidas y rechace las prohibidas.
test05	Rangos de η y λ	Comprueba el rechazo de valores no finitos o fuera de dominio (η ∉ [0, 1], λ < 0).
test06	Extremos RCL	Valida el comportamiento de la RCL en η = 0 (goloso) y η = 1 (aleatorio) con semilla fija.
test07	Reconstrucción c_ij	Verifica la actualización de la ubicación previa p_i^{t+1} y reconstrucción de costos.
test08	Geometría O_j	Comprueba el cálculo exacto de lados exteriores para celdas interior, borde y esquina.
test09	Patrón Grises G_i	Valida que las piezas tengan como máximo 2 lados grises y que sean estrictamente adyacentes.
test10	Validador Z_t	Comprueba que Validator declare inválidas asignaciones con sobreocupación o desencajes.

7. Bibliografía

[1] Monckton, C. (2007). Eternity II, commercial edge-matching puzzle. TOMY UK Ltd.

[2] Demaine, E. D., & Demaine, M. L. (2007). Jigsaw Puzzles, Edge Matching, and Polyomino Packing: Connections and Complexity. Graphs and Combinatorics, 23(1), 195-208.

[3] Land, A. H., & Doig, A. G. (1960). An Automatic Method of Solving Discrete Programming Problems. Econometrica, 28(3), 497-520.

[4] Forrest, J., de la Nuez, D., & Lougee-Heimer, R. (2026). CLP User's Guide. COIN-OR Project. https://coin-or.github.io/Clp/

[5] Feo, T. A., & Resende, M. G. (1995). Greedy Randomized Adaptive Search Procedures. Journal of Global Optimization, 6, 109-133.

[6] Garey, M. R., & Johnson, D. S. (1979). Computers and Intractability: A Guide to the Theory of NP-Completeness. W. H. Freeman.

[7] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.